SQL Easy: documentación de las consultas

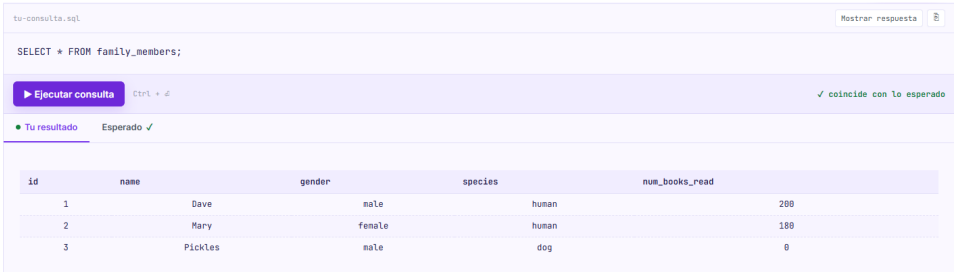
SQL & Web Security Challenge · HackerWomen Council

Autora: Sofia

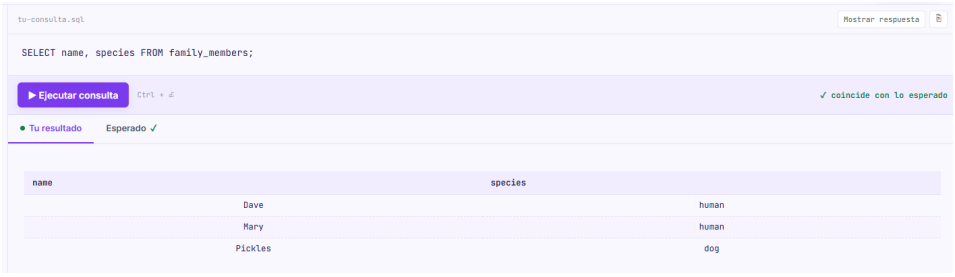
Ejercicios 1 a 31 de 31. Captura del ejercicio 22 pendiente de insertar.

Fundamentos (ejercicios 1 a 11)

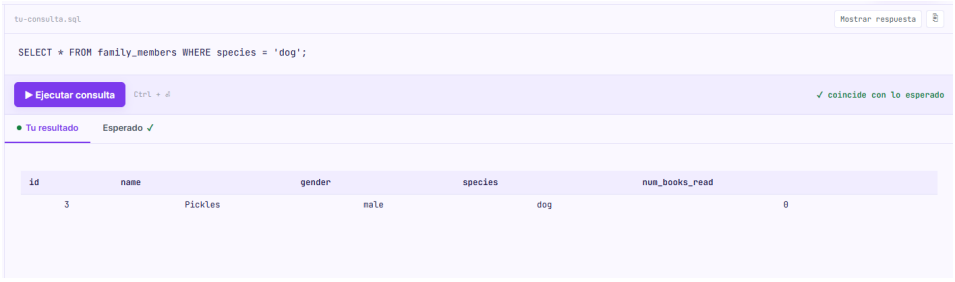
Ejercicio 1: SELECT *

01 Ejercicio	Ejercicio 1: SELECT *
02 Objetivo	Obtener todos los datos de la tabla family_members.
03 Concepto SQL	SELECT con * devuelve todas las columnas de una tabla.
04 Consulta ejecutada	`SELECT * FROM family_members;`
05 Resultado	3 filas: Dave, Mary y Pickles 
06 Interpretación	Devolvió las 3 filas de family_members con sus 5 columnas: id, name, gender, species y num_books_read.
07 Aprendizaje	SELECT indica qué columnas ver y FROM de qué tabla. El * evita listar cada columna.
08 Aplicación en el mundo real	Revisar rápido el contenido de una tabla, por ejemplo una lista de clientes, antes de filtrar o analizar.

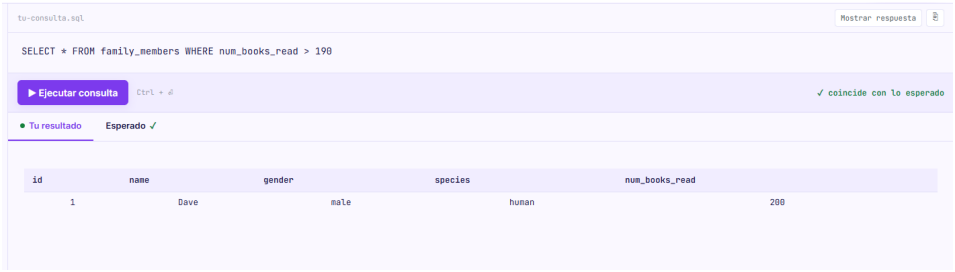
Ejercicio 2: SELECT columnas específicas

01 Ejercicio	Ejercicio 2: SELECT columnas específicas
02 Objetivo	Devolver solo las columnas name y species de family_members.
03 Concepto SQL	SELECT con nombres de columna devuelve solo esos campos.
04 Consulta ejecutada	`SELECT name, species FROM family_members;`
05 Resultado	3 filas: Dave/human, Mary/human, Pickles/dog 
06 Interpretación	Devuelve las 3 filas (Dave, Mary, Pickles) solo con nombre y especie.
07 Aprendizaje	Se eligen columnas separándolas con comas. Primero probé el ejemplo del enunciado y el resultado no coincidía, así que releí qué pedía.
08 Aplicación en el mundo real	Sacar solo nombre y categoría de una tabla de productos sin traer el resto de los datos.

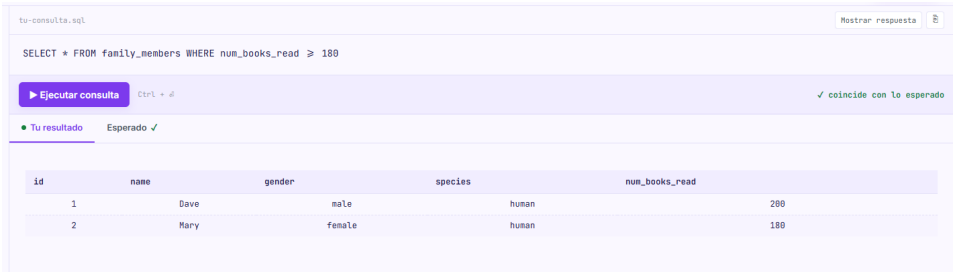
Ejercicio 3: WHERE ... Equals

01 Ejercicio	Ejercicio 3: WHERE ... Equals
02 Objetivo	Devolver solo las filas donde la especie es perro.
03 Concepto SQL	WHERE filtra filas con una condición de igualdad (=).
04 Consulta ejecutada	`SELECT * FROM family_members WHERE species = 'dog';`
05 Resultado	1 fila: Pickles 
06 Interpretación	Devuelve 1 fila: Pickles (id 3, male, dog, 0 libros).
07 Aprendizaje	Los valores de texto van entre comillas simples; el comando WHERE no.
08 Aplicación en el mundo real	Filtrar los pedidos de un cliente concreto o los productos de una categoría.

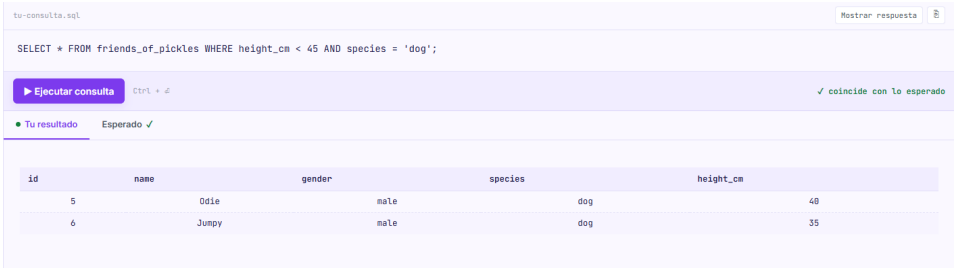
Ejercicio 4: WHERE ... Mayor que

01 Ejercicio	Ejercicio 4: WHERE ... Mayor que
02 Objetivo	Devolver las filas de miembros con num_books_read mayor que 190.
03 Concepto SQL	WHERE con el operador > filtra valores numéricos estrictamente mayores.
04 Consulta ejecutada	`SELECT * FROM family_members WHERE num_books_read > 190`
05 Resultado	1 fila: Dave, 200 libros 
06 Interpretación	Solo Dave supera los 190 libros. Mary (180) y Pickles (0) quedan fuera.
07 Aprendizaje	> compara números sin comillas y excluye el valor exacto.
08 Aplicación en el mundo real	Encontrar clientes con compras superiores a cierto monto.

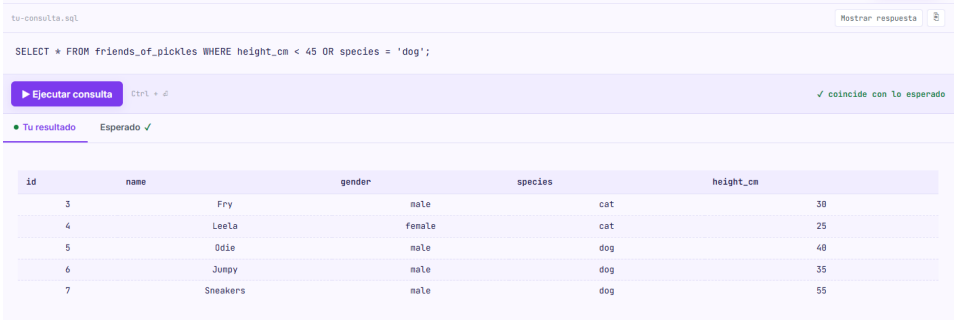
Ejercicio 5: WHERE ... Mayor o igual que

01 Ejercicio	Ejercicio 5: WHERE ... Mayor o igual que
02 Objetivo	Devolver las filas de family_members donde num_books_read es mayor o igual a 180.
03 Concepto SQL	WHERE con el operador >= incluye el valor exacto.
04 Consulta ejecutada	`SELECT * FROM family_members WHERE num_books_read >= 180;`
05 Resultado	2 filas: Dave (200) y Mary (180) 
06 Interpretación	Devuelve 2 filas: Dave (200) y Mary (180). Pickles (0) queda fuera.
07 Aprendizaje	A diferencia de >, el >= incluye el 180. Mi primer intento falló por omitir * FROM; el error "no such column" me ayudó a ubicarlo.
08 Aplicación en el mundo real	Seleccionar clientes con compras de al menos un monto mínimo para un programa de beneficios.

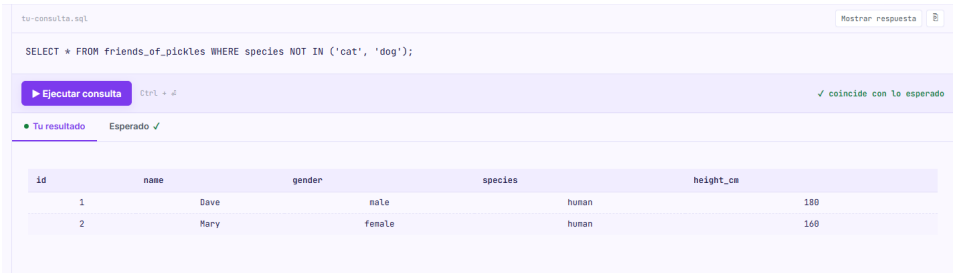
Ejercicio 6: AND

01 Ejercicio	Ejercicio 6: AND
02 Objetivo	Encontrar los amigos de Pickles que son perros y miden menos de 45 cm.
03 Concepto SQL	AND combina condiciones en WHERE: todas deben cumplirse.
04 Consulta ejecutada	`SELECT * FROM friends_of_pickles WHERE height_cm < 45 AND species = 'dog';`
05 Resultado	2 filas: Odie (40 cm) y Jumpy (35 cm) 
06 Interpretación	Devuelve 2 filas: Odie (40 cm) y Jumpy (35 cm). Sneakers (55 cm) queda fuera por su altura.
07 Aprendizaje	Con AND cada fila debe cumplir ambas condiciones. Mi primer intento usó > en vez de <, y devolvió a Sneakers, que no cumple lo pedido.
08 Aplicación en el mundo real	Filtrar productos de una categoría y por debajo de cierto precio.

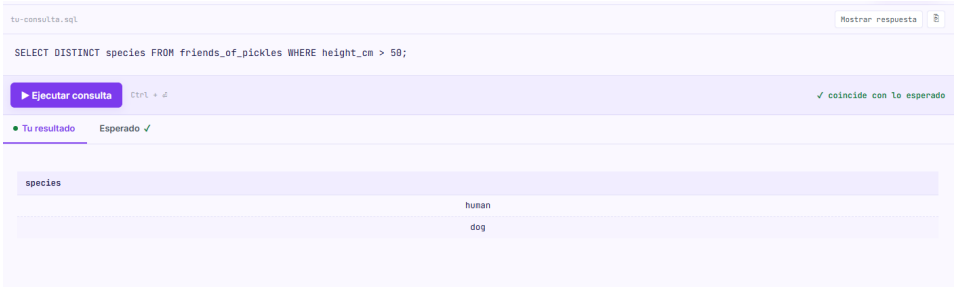
Ejercicio 7: OR

01 Ejercicio	Ejercicio 7: OR
02 Objetivo	Encontrar los amigos de Pickles que son perros o miden menos de 45 cm.
03 Concepto SQL	OR en WHERE devuelve las filas que cumplan al menos una de las condiciones.
04 Consulta ejecutada	`SELECT * FROM friends_of_pickles WHERE height_cm < 45 OR species = 'dog';`
05 Resultado	5 filas: Fry, Leela, Odie, Jumpy y Sneakers 
06 Interpretación	Fry y Leela entran por su altura (30 y 25 cm); Odie, Jumpy y Sneakers por ser perros. Dave y Mary no cumplen ninguna.
07 Aprendizaje	OR amplía el resultado, mientras que AND lo restringe. Con el ejercicio 6 la misma pareja de condiciones devolvía 2 filas y aquí 5.
08 Aplicación en el mundo real	Buscar clientes de cierta ciudad o con compras recientes, para una campaña.

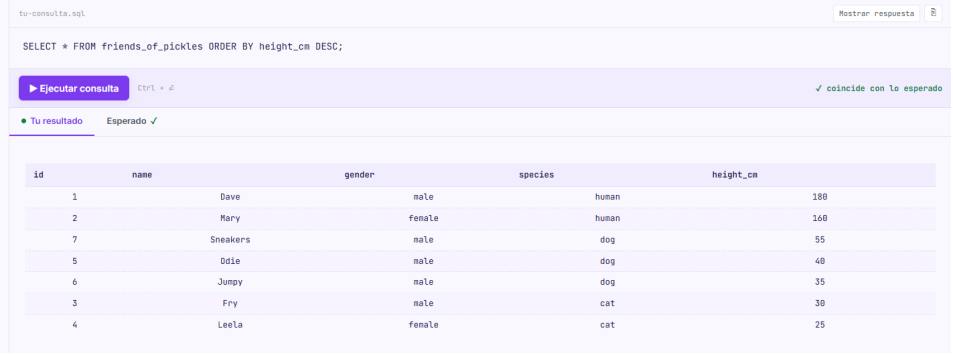
Ejercicio 8: IN

01 Ejercicio	Ejercicio 8: IN
02 Objetivo	Devolver las filas de friends_of_pickles que no son gatos ni perros.
03 Concepto SQL	IN busca valores dentro de una lista, y NOT IN los que no están en ella.
04 Consulta ejecutada	`SELECT * FROM friends_of_pickles WHERE species NOT IN ('cat', 'dog');`
05 Resultado	2 filas: Dave y Mary 
06 Interpretación	Solo Dave y Mary quedan, porque son humanos. Los gatos (Fry, Leela) y los perros (Odie, Jumpy, Sneakers) se excluyen.
07 Aprendizaje	NOT IN ('cat', 'dog') reemplaza varias condiciones con OR y es más corto y legible.
08 Aplicación en el mundo real	Excluir varias categorías a la vez, como pedidos cancelados o devueltos en un reporte de ventas.

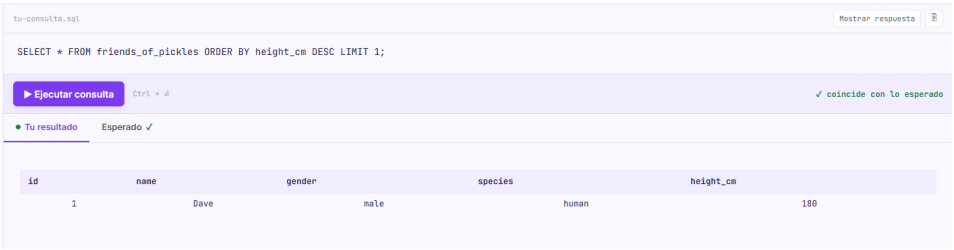
Ejercicio 9: DISTINCT

01 Ejercicio	Ejercicio 9: DISTINCT
02 Objetivo	Devolver la lista de especies distintas de los animales con más de 50 cm de altura.
03 Concepto SQL	DISTINCT elimina las filas duplicadas del resultado.
04 Consulta ejecutada	`SELECT DISTINCT species FROM friends_of_pickles WHERE height_cm > 50;`
05 Resultado	2 filas: human y dog 
06 Interpretación	Devuelve 2 filas: human y dog. Dave, Mary y Sneakers superan los 50 cm, pero human se muestra una sola vez.
07 Aprendizaje	DISTINCT aplica a todas las columnas del SELECT. Mi primer intento incluyó gender y devolvió 3 combinaciones en vez de 2 especies.
08 Aplicación en el mundo real	Obtener la lista de países o categorías distintas que aparecen en una tabla de ventas.

Ejercicio 10: ORDER BY

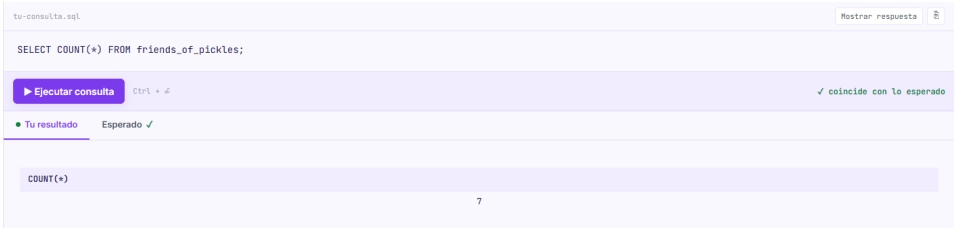
01 Ejercicio	Ejercicio 10: ORDER BY
02 Objetivo	Ordenar los friends_of_pickles por height_cm de mayor a menor.
03 Concepto SQL	ORDER BY ordena el resultado por una columna, y DESC lo hace en orden descendente.
04 Consulta ejecutada	`SELECT * FROM friends_of_pickles ORDER BY height_cm DESC;`
05 Resultado	7 filas, de Dave (180) a Leela (25) 
06 Interpretación	Devuelve las 7 filas de la más alta a la más baja: Dave (180), Mary (160), Sneakers (55), Odie (40), Jumpy (35), Fry (30) y Leela (25).
07 Aprendizaje	Sin DESC el orden es ascendente por defecto. ORDER BY va al final de la consulta.
08 Aplicación en el mundo real	Mostrar los productos más caros primero o las ventas más recientes arriba en un reporte.

Ejercicio 11: LIMIT # de filas devueltas

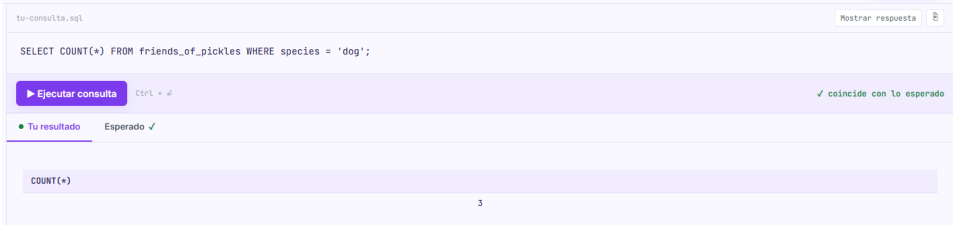
01 Ejercicio	Ejercicio 11: LIMIT # de filas devueltas
02 Objetivo	Devolver la fila única, con todas sus columnas, del amigo de Pickles más alto.
03 Concepto SQL	LIMIT restringe cuántas filas devuelve la consulta. Combinado con ORDER BY, trae las primeras del orden elegido.
04 Consulta ejecutada	`SELECT * FROM friends_of_pickles ORDER BY height_cm DESC LIMIT 1;`
05 Resultado	1 fila: Dave, 180 cm 
06 Interpretación	Devuelve 1 fila: Dave (id 1, male, human, 180 cm).
07 Aprendizaje	LIMIT va después de ORDER BY ... DESC. Ordenar de mayor a menor y limitar a 1 da el valor máximo. Algunas variantes de SQL no usan LIMIT.
08 Aplicación en el mundo real	Ver el producto más caro o la última venta registrada sin cargar toda la tabla.

Agregaciones (ejercicios 12 a 20)

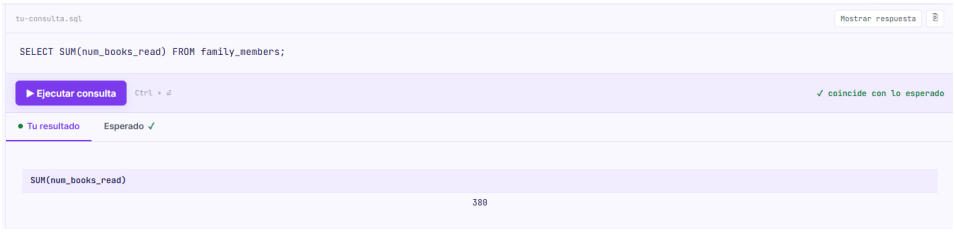
Ejercicio 12: COUNT(*)

01 Ejercicio	Ejercicio 12: COUNT(*)
02 Objetivo	Obtener el número total de filas de la tabla friends_of_pickles.
03 Concepto SQL	COUNT(*) es una función de agregación que cuenta las filas.
04 Consulta ejecutada	`SELECT COUNT(*) FROM friends_of_pickles;`
05 Resultado	Resultado: 7 
06 Interpretación	La tabla tiene 7 filas, una por cada amigo de Pickles.
07 Aprendizaje	Una función de agregación devuelve un solo valor en lugar de listar las filas. Sirve para verificar el tamaño de una tabla.
08 Aplicación en el mundo real	Saber cuántos clientes, pedidos o registros hay en una tabla, o verificar que una carga de datos no perdió filas.

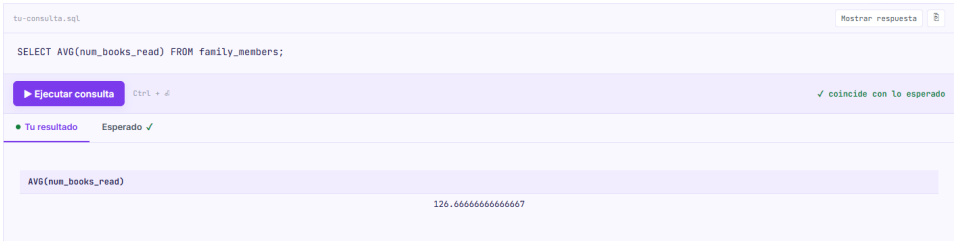
Ejercicio 13: COUNT(*) ... WHERE

01 Ejercicio	Ejercicio 13: COUNT(*) ... WHERE
02 Objetivo	Contar las filas de friends_of_pickles donde la especie es perro.
03 Concepto SQL	COUNT(*) combinado con WHERE cuenta solo las filas que cumplen la condición.
04 Consulta ejecutada	`SELECT COUNT(*) FROM friends_of_pickles WHERE species = 'dog';`
05 Resultado	Resultado: 3 
06 Interpretación	Hay 3 perros en la tabla: Odie, Jumpy y Sneakers.
07 Aprendizaje	WHERE filtra primero y COUNT(*) cuenta después solo las filas filtradas, no las 7 de la tabla.
08 Aplicación en el mundo real	Contar los pedidos pendientes o los clientes de una ciudad concreta.

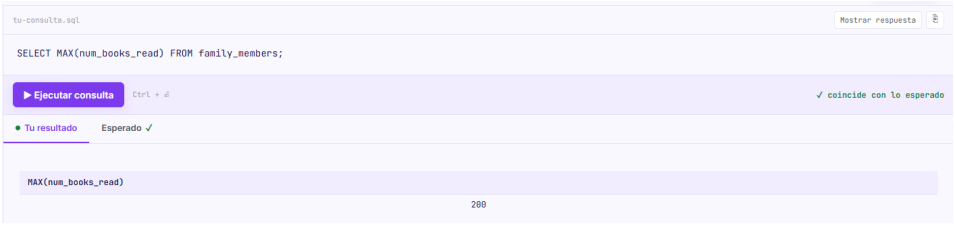
Ejercicio 14: SUM

01 Ejercicio	Ejercicio 14: SUM
02 Objetivo	Encontrar el total de libros leídos (num_books_read) por la familia.
03 Concepto SQL	SUM es una función de agregación que suma los valores de una columna numérica.
04 Consulta ejecutada	`SELECT SUM(num_books_read) FROM family_members;`
05 Resultado	Resultado: 380 
06 Interpretación	Dave (200) + Mary (180) + Pickles (0) = 380 libros leídos en total.
07 Aprendizaje	SUM devuelve un solo valor y funciona con columnas numéricas. Se puede combinar con WHERE para sumar solo algunas filas.
08 Aplicación en el mundo real	Calcular el total de ventas de un mes o el monto total de facturas de un cliente.

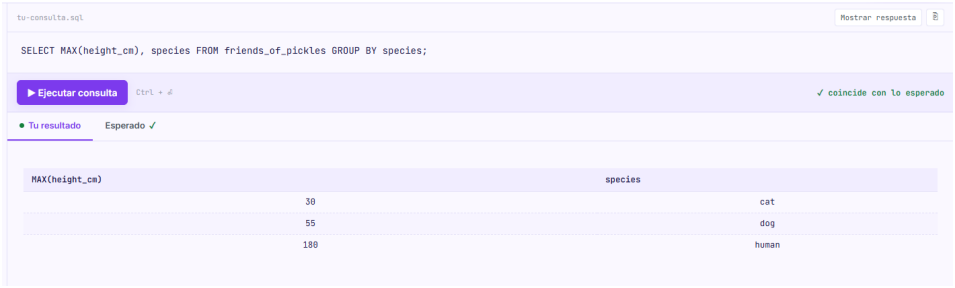
Ejercicio 15: AVG

01 Ejercicio	Ejercicio 15: AVG
02 Objetivo	Encontrar el promedio de libros leídos (num_books_read) por cada miembro de la familia.
03 Concepto SQL	AVG es una función de agregación que calcula el promedio de una columna numérica.
04 Consulta ejecutada	`SELECT AVG(num_books_read) FROM family_members;`
05 Resultado	Resultado: 126.66666666666667 
06 Interpretación	(200 + 180 + 0) ÷ 3 = 126.67 libros por miembro. Pickles entra al cálculo con 0.
07 Aprendizaje	AVG divide la suma entre la cantidad de filas, e incluye los ceros. El decimal largo se debe a cómo las computadoras manejan los números.
08 Aplicación en el mundo real	Calcular el ticket promedio de compra o la nota media de un curso.

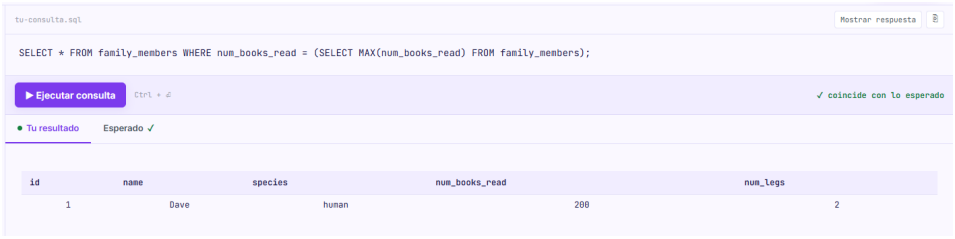
Ejercicio 16: MAX

01 Ejercicio	Ejercicio 16: MAX
02 Objetivo	Encontrar el mayor número de libros leídos (num_books_read) por un miembro de la familia.
03 Concepto SQL	MAX y MIN son funciones de agregación que devuelven el valor máximo o mínimo de una columna.
04 Consulta ejecutada	`SELECT MAX(num_books_read) FROM family_members;`
05 Resultado	Resultado: 200 
06 Interpretación	El máximo es 200 libros, de Dave. Mary leyó 180 y Pickles 0.
07 Aprendizaje	MAX devuelve solo el valor, no la fila completa. MIN funciona igual para el valor más bajo.
08 Aplicación en el mundo real	Encontrar la venta más alta del mes o el precio más bajo de un producto.

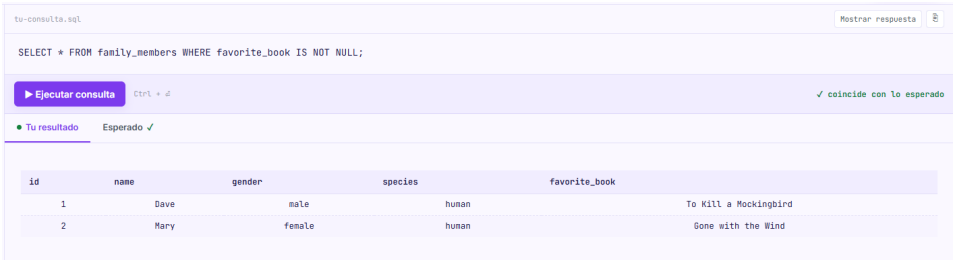
Ejercicio 17: GROUP BY

01 Ejercicio	Ejercicio 17: GROUP BY
02 Objetivo	Devolver la altura más alta de cada especie, junto con el nombre de la especie.
03 Concepto SQL	GROUP BY divide la tabla en grupos según el valor de una columna. La función de agregación se calcula por cada grupo.
04 Consulta ejecutada	`SELECT MAX(height_cm), species FROM friends_of_pickles GROUP BY species;`
05 Resultado	3 filas: cat 30, dog 55 y human 180 
06 Interpretación	Devuelve una fila por especie con su altura máxima: Fry mide 30 cm entre los gatos, Sneakers 55 cm entre los perros y Dave 180 cm entre los humanos.
07 Aprendizaje	La columna por la que se agrupa (species) también debe ir en el SELECT para saber a qué grupo pertenece cada valor.
08 Aplicación en el mundo real	Ver las ventas totales por región o el pedido más grande de cada cliente.

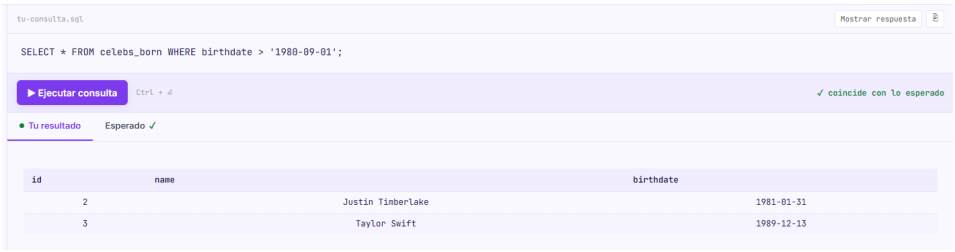
Ejercicio 18: Consultas anidadas

01 Ejercicio	Ejercicio 18: Consultas anidadas
02 Objetivo	Devolver los miembros de la familia con el mayor número de libros leídos (num_books_read).
03 Concepto SQL	Una consulta anidada (subconsulta) es un SELECT dentro de otro. La interior se ejecuta primero y su resultado se usa en la exterior.
04 Consulta ejecutada	`SELECT * FROM family_members WHERE num_books_read = (SELECT MAX(num_books_read) FROM family_members);`
05 Resultado	1 fila: Dave, 200 libros 
06 Interpretación	La subconsulta devuelve 200 (el máximo) y la consulta exterior trae la fila con ese valor: Dave (id 1, human, 200 libros, 2 patas).
07 Aprendizaje	Con MAX solo obtenía el número. La subconsulta permite traer la fila completa que tiene ese valor.
08 Aplicación en el mundo real	Encontrar al cliente con la compra más alta, o los productos con el precio mínimo de su tabla.

Ejercicio 19: NULL

01 Ejercicio	Ejercicio 19: NULL
02 Objetivo	Devolver las filas de family_members donde favorite_book no es nulo.
03 Concepto SQL	NULL indica que una columna no tiene ningún valor. Se filtra con IS NULL o IS NOT NULL.
04 Consulta ejecutada	`SELECT * FROM family_members WHERE favorite_book IS NOT NULL;`
05 Resultado	2 filas: Dave y Mary 
06 Interpretación	Devuelve a Dave (To Kill a Mockingbird) y a Mary (Gone with the Wind). Pickles queda fuera porque su favorite_book es NULL.
07 Aprendizaje	NULL no es un valor, es la ausencia de uno, así que no se compara con = sino con IS NULL o IS NOT NULL.
08 Aplicación en el mundo real	Encontrar clientes sin correo registrado o pedidos sin fecha de entrega, para completar esos datos.

Ejercicio 20: Fecha

01 Ejercicio	Ejercicio 20: Fecha
02 Objetivo	Devolver las celebridades de celebs_born que nacieron después del 1 de septiembre de 1980.
03 Concepto SQL	Las fechas se guardan como AAAA-MM-DD y se comparan con < y > igual que los números, escritas entre comillas simples.
04 Consulta ejecutada	`SELECT * FROM celebs_born WHERE birthdate > '1980-09-01';`
05 Resultado	2 filas: Justin Timberlake (1981-01-31) y Taylor Swift (1989-12-13) 
06 Interpretación	Devuelve 2 filas con nacimiento posterior al 1 de septiembre de 1980: Justin Timberlake (1981-01-31) y Taylor Swift (1989-12-13).
07 Aprendizaje	Las fechas se comparan en orden de año, mes y día. El operador > excluye el 1 de septiembre de 1980; para incluirlo habría que usar >=.
08 Aplicación en el mundo real	Filtrar pedidos o registros posteriores a cierta fecha, como las ventas del último trimestre.

Joins (ejercicios 21 a 27)

Ejercicio 21: Inner joins

01 Ejercicio	Ejercicio 21: Inner joins
02 Objetivo	Emparejar cada personaje con el actor que lo interpreta, mostrando character.name y character_actor.actor_name.
03 Concepto SQL	INNER JOIN ... ON une filas de dos tablas que comparten una columna de referencia: aquí character.id y character_actor.character_id.

04 Consulta ejecutada	`SELECT character.name, character_actor.actor_name`
	`FROM character`
	`INNER JOIN character_actor`
	`ON character.id = character_actor.character_id;`

05 Resultado	4 filas con name y actor_name • Tu resultado Esperado ✓ name | actor_name Doogie Howser | Neil Patrick Harris Barney Stinson | Neil Patrick Harris Lily Aldrin | Alyson Hannigan Willow Rosenberg | Alyson Hannigan
06 Interpretación	Devuelve 4 filas: Doogie Howser y Barney Stinson con Neil Patrick Harris, y Lily Aldrin y Willow Rosenberg con Alyson Hannigan. Un mismo actor aparece en dos filas porque interpreta a dos personajes.
07 Aprendizaje	Los datos se guardan en tablas separadas para no duplicarlos y INNER JOIN las reúne por el identificador. Se escribe tabla.columna para evitar ambigüedad entre tablas.
08 Aplicación en el mundo real	Unir una tabla de pedidos con la de clientes para ver el nombre del cliente junto a cada pedido.

Ejercicio 22: Múltiples joins

01 Ejercicio	Ejercicio 22: Múltiples joins
02 Objetivo	Emparejar el nombre de cada personaje con el de su actor usando dos joins y la tabla de relación character_actor. Columnas: character.name, actor.name.
03 Concepto SQL	Varios INNER JOIN encadenados conectan tres tablas: una entidad, la tabla de relación y la otra entidad.

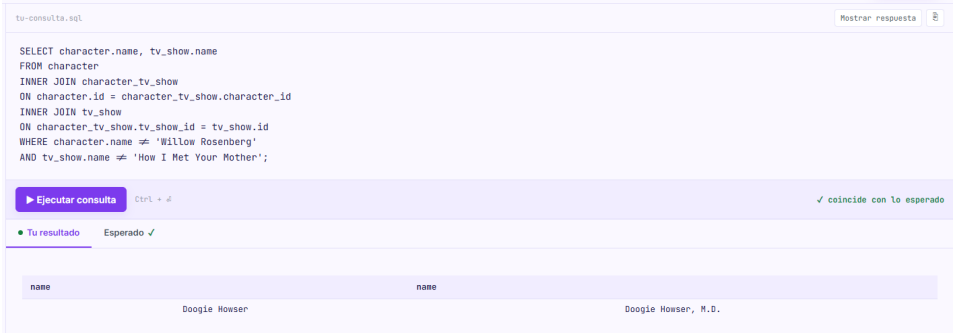
04 Consulta ejecutada	`SELECT character.name, actor.name`
	`FROM character`
	`INNER JOIN character_actor`
	`ON character.id = character_actor.character_id`
	`INNER JOIN actor`
	`ON character_actor.actor_id = actor.id;`

05 Resultado	*[Inserta aquí tu captura]*
06 Interpretación	La consulta devuelve cada personaje junto al actor que lo interpreta, pasando por la tabla de relación character_actor.
07 Aprendizaje	La tabla de relación no tiene nombres, solo ids, y sirve de puente entre character y actor. Así ningún nombre se duplica. Los joins se leen en orden: primero personaje con relación y luego relación con actor.
08 Aplicación en el mundo real	Mostrar el nombre del cliente y el del producto de un pedido, pasando por la tabla de detalle de pedidos.

Ejercicio 23: Joins con WHERE

01 Ejercicio	Ejercicio 23: Joins con WHERE
02 Objetivo	Devolver personajes y programas de TV, excluyendo al personaje «Willow Rosenberg» y el programa «How I Met Your Mother».
03 Concepto SQL	WHERE filtra después de unir las tablas con INNER JOIN. Se puede condicionar por columnas de cualquiera de las tablas unidas, usando tabla.columna.

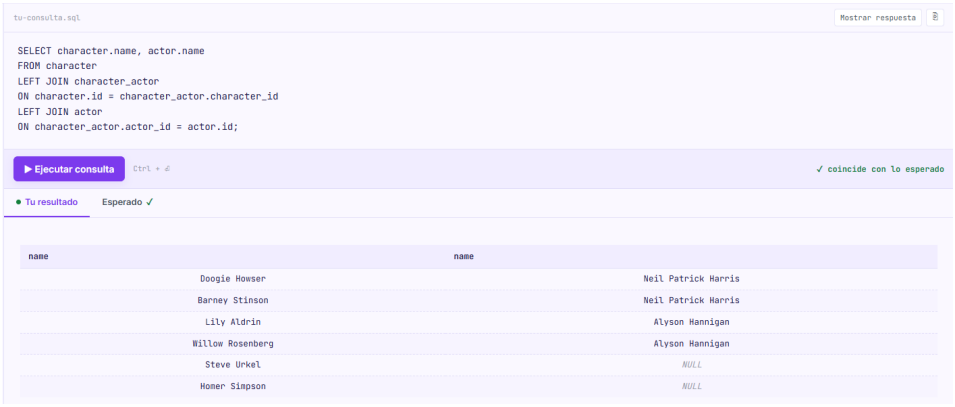
04 Consulta ejecutada	`SELECT character.name, tv_show.name`
	`FROM character`
	`INNER JOIN character_tv_show`
	`ON character.id = character_tv_show.character_id`
	`INNER JOIN tv_show`
	`ON character_tv_show.tv_show_id = tv_show.id`
	`WHERE character.name != 'Willow Rosenberg'`
	`AND tv_show.name != 'How I Met Your Mother';`

05 Resultado	Primera fila visible: Doogie Howser y «Doogie Howser, M.D.» 
06 Interpretación	La captura muestra el resultado con las columnas name y name (personaje y programa); la primera fila visible es Doogie Howser en «Doogie Howser, M.D.». Willow Rosenberg y el programa How I Met Your Mother quedan excluidos por el WHERE.
07 Aprendizaje	WHERE va al final, después de todos los joins. El operador != excluye un valor y AND exige que se cumplan ambas condiciones.
08 Aplicación en el mundo real	Listar pedidos con el nombre del cliente, excluyendo a cierto cliente o los pedidos cancelados.

Ejercicio 24: Left joins

01 Ejercicio	Ejercicio 24: Left joins
02 Objetivo	Obtener todos los personajes con el actor que los interpreta, incluyendo los que no tienen actor. Columnas: character.name, actor.name.
03 Concepto SQL	LEFT JOIN devuelve todas las filas de la tabla izquierda y completa con NULL las columnas de la derecha cuando no hay coincidencia.

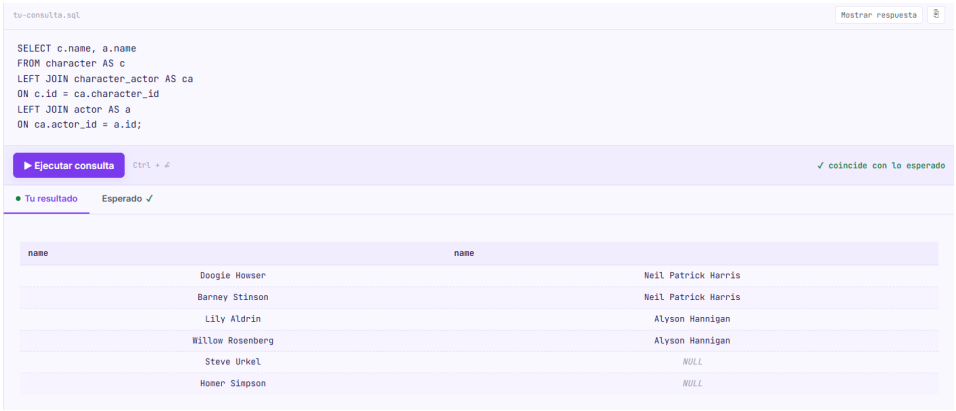
04 Consulta ejecutada	`SELECT character.name, actor.name`
	`FROM character`
	`LEFT JOIN character_actor`
	`ON character.id = character_actor.character_id`
	`LEFT JOIN actor`
	`ON character_actor.actor_id = actor.id;`

05 Resultado	6 filas: 4 personajes con actor y 2 con NULL 
06 Interpretación	Devuelve 6 filas. Doogie Howser y Barney Stinson aparecen con Neil Patrick Harris, y Lily Aldrin y Willow Rosenberg con Alyson Hannigan. Steve Urkel y Homer Simpson no tienen actor asociado, por eso su actor sale como NULL.
07 Aprendizaje	INNER JOIN solo trae filas con datos en ambas tablas, mientras que LEFT JOIN conserva todas las de la tabla izquierda. SQLite no tiene RIGHT JOIN ni OUTER JOIN.
08 Aplicación en el mundo real	Listar todos los clientes, incluso los que nunca han hecho un pedido, para detectar clientes inactivos.

Ejercicio 25: Alias de tabla

01 Ejercicio	Ejercicio 25: Alias de tabla
02 Objetivo	Emparejar cada personaje con su actor usando LEFT JOIN y los alias c, ca y a. Columnas: c.name, a.name.
03 Concepto SQL	AS crea un alias (apodo) para una tabla, lo que acorta las consultas con varios joins.

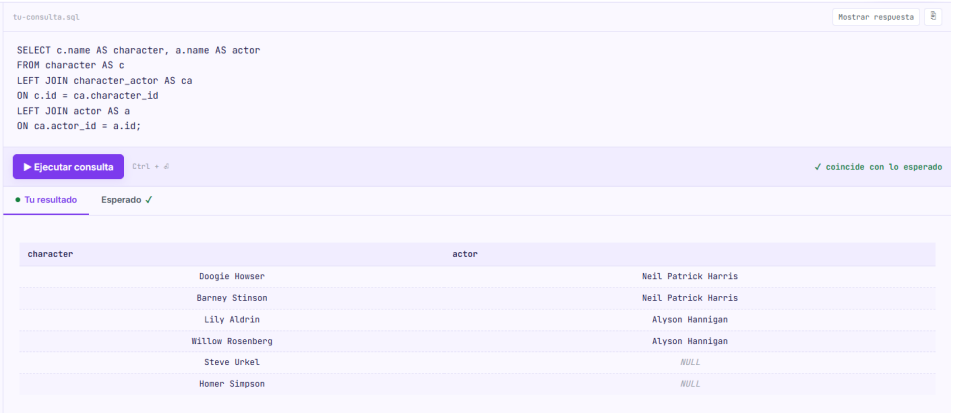
04 Consulta ejecutada	`SELECT c.name, a.name`
	`FROM character AS c`
	`LEFT JOIN character_actor AS ca`
	`ON c.id = ca.character_id`
	`LEFT JOIN actor AS a`
	`ON ca.actor_id = a.id;`

05 Resultado	6 filas, igual que el ejercicio 24 
06 Interpretación	Devuelve las mismas 6 filas que el ejercicio 24, incluidos Steve Urkel y Homer Simpson con NULL, pero con una consulta más corta gracias a los alias c, ca y a.
07 Aprendizaje	Una vez definido el alias, se usa en el resto de la consulta, incluso en SELECT, que se escribe antes pero se evalúa después. Los alias no cambian el resultado, solo la legibilidad.
08 Aplicación en el mundo real	En reportes con muchas tablas, como pedidos, clientes y productos, los alias evitan consultas largas y difíciles de leer.

Ejercicio 26: Alias de columna

01 Ejercicio	Ejercicio 26: Alias de columna
02 Objetivo	Emparejar cada personaje con su actor usando LEFT JOIN y nombrar las dos columnas del resultado como character y actor.
03 Concepto SQL	AS también crea alias para columnas y cambia el nombre con que aparecen en el resultado.

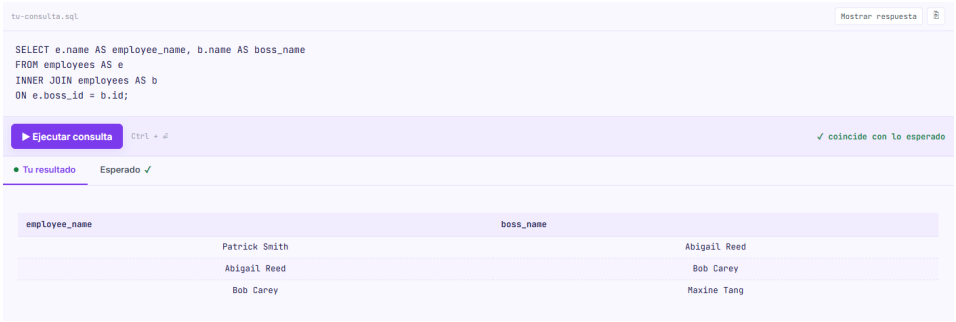
04 Consulta ejecutada	`SELECT c.name AS character, a.name AS actor`
	`FROM character AS c`
	`LEFT JOIN character_actor AS ca`
	`ON c.id = ca.character_id`
	`LEFT JOIN actor AS a`
	`ON ca.actor_id = a.id;`

05 Resultado	6 filas con las columnas character y actor 
06 Interpretación	Devuelve las mismas 6 filas que el ejercicio 25, pero ahora los encabezados son character y actor en lugar de dos columnas llamadas name. Steve Urkel y Homer Simpson siguen con NULL.
07 Aprendizaje	En el ejercicio anterior ambas columnas se llamaban name, lo que confundía. Con alias queda claro qué contiene cada una. El alias cambia el encabezado, no los datos.
08 Aplicación en el mundo real	Poner encabezados legibles en reportes, como «Cliente» y «Total de ventas», en lugar de nombres técnicos de columna.

Ejercicio 27: Self joins

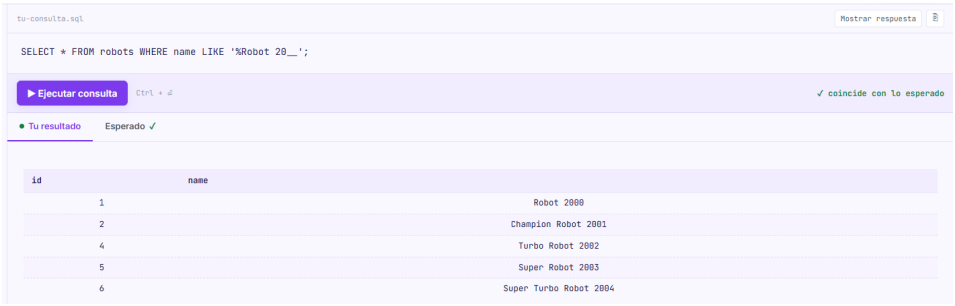
01 Ejercicio	Ejercicio 27: Self joins
02 Objetivo	Devolver el nombre de cada empleado junto al de su jefe, con las columnas employee_name y boss_name.
03 Concepto SQL	Un self join une una tabla consigo misma. Requiere alias de tabla distintos para distinguir cuál es la versión de empleado y cuál la de jefe.

04 Consulta ejecutada	`SELECT e.name AS employee_name, b.name AS boss_name`
	`FROM employees AS e`
	`INNER JOIN employees AS b`
	`ON e.boss_id = b.id;`

05 Resultado	3 filas con employee_name y boss_name 
06 Interpretación	Devuelve 3 filas: Patrick Smith con jefa Abigail Reed, Abigail Reed con jefe Bob Carey y Bob Carey con jefa Maxine Tang. Se ve la cadena jerárquica; Maxine Tang aparece solo como jefa.
07 Aprendizaje	La misma tabla cumple dos roles y los alias e y b permiten distinguirlos. La relación jefe–empleado se guarda como un id que apunta a otra fila de la misma tabla.
08 Aplicación en el mundo real	Mostrar la jerarquía de una empresa, o categorías con subcategorías en una tienda en línea.

Texto y condiciones (ejercicios 28 a 31)

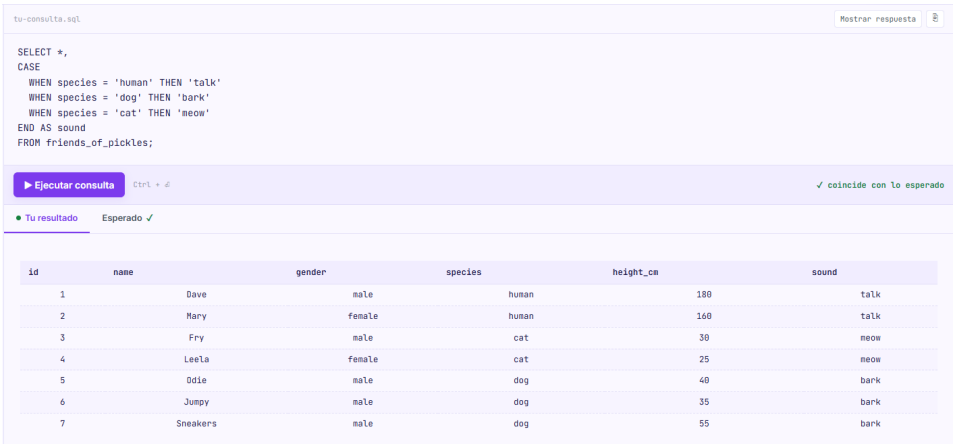
Ejercicio 28: LIKE

01 Ejercicio	Ejercicio 28: LIKE
02 Objetivo	Devolver los robots cuyo nombre termina en «Robot» seguido de un año entre 2000 y 2099.
03 Concepto SQL	LIKE busca patrones en texto. % representa cero o más caracteres y _ representa exactamente uno.
04 Consulta ejecutada	`SELECT * FROM robots WHERE name LIKE '%Robot 20__';`
05 Resultado	5 filas: Robot 2000, Champion Robot 2001, Turbo Robot 2002, Super Robot 2003 y Super Turbo Robot 2004 
06 Interpretación	Devuelve 5 filas (ids 1, 2, 4, 5 y 6). Todas terminan en «Robot» más un año entre 2000 y 2004. La fila con id 3 queda fuera porque su nombre no cumple el patrón.
07 Aprendizaje	Como los nombres pueden tener texto antes de «Robot» (por ejemplo «Champion Robot 2001»), el patrón empieza con %. Cada _ cuenta un carácter exacto, por eso 20__ limita el año a 2000–2099.
08 Aplicación en el mundo real	Buscar clientes cuyo correo termina en cierto dominio, o códigos de producto con un formato fijo como «SKU-2024-A».

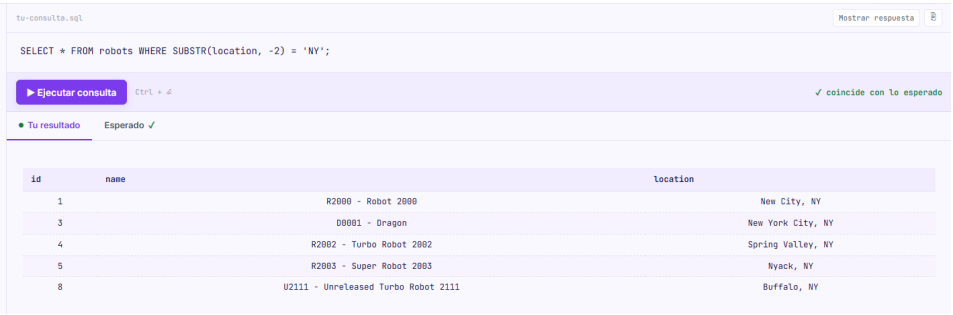
Ejercicio 29: CASE

01 Ejercicio	Ejercicio 29: CASE
02 Objetivo	Devolver todas las columnas de friends_of_pickles más una columna sound con «talk» para humanos, «bark» para perros y «meow» para gatos.
03 Concepto SQL	CASE WHEN ... THEN ... END devuelve un valor distinto según la condición que se cumpla, y AS le da nombre a la columna resultante.

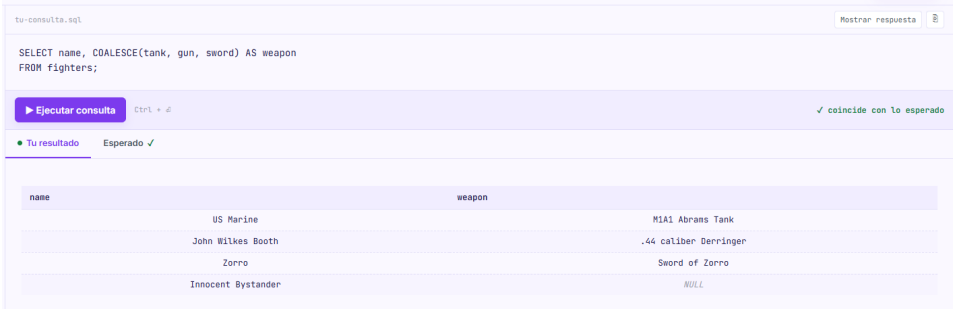
04 Consulta ejecutada	`SELECT *,`
	`CASE`
	`  WHEN species = 'human' THEN 'talk'`
	`  WHEN species = 'dog' THEN 'bark'`
	`  WHEN species = 'cat' THEN 'meow'`
	`END AS sound`
	`FROM friends_of_pickles;`

05 Resultado	7 filas, con la columna sound 
06 Interpretación	Devuelve las 7 filas de friends_of_pickles con la columna sound: Dave y Mary con «talk», Fry y Leela con «meow», y Odie, Jumpy y Sneakers con «bark».
07 Aprendizaje	CASE crea una columna nueva calculada, sin modificar la tabla. Los WHEN se evalúan en orden y ELSE cubriría los casos restantes.
08 Aplicación en el mundo real	Clasificar clientes según su nivel de compras, como «Bronce», «Plata» u «Oro», o traducir códigos de estado a texto legible en un reporte.

Ejercicio 30: SUBSTR

01 Ejercicio	Ejercicio 30: SUBSTR
02 Objetivo	Devolver todos los robots ubicados en NY.
03 Concepto SQL	SUBSTR(columna, índice, largo) extrae una parte de un texto. Un índice negativo cuenta desde el final de la cadena.
04 Consulta ejecutada	`SELECT * FROM robots WHERE SUBSTR(location, -2) = 'NY';`
05 Resultado	5 filas: ids 1, 3, 4, 5 y 8 
06 Interpretación	Devuelve 5 robots ubicados en NY: R2000 - Robot 2000 (New City), D0001 - Dragon (New York City), R2002 - Turbo Robot 2002 (Spring Valley), R2003 - Super Robot 2003 (Nyack) y U2111 - Unreleased Turbo Robot 2111 (Buffalo).
07 Aprendizaje	Con -2 se obtienen los últimos dos caracteres, el estado, sin importar la ciudad ni su largo. Es una alternativa a LIKE '%NY'. En otros SQL se usaría RIGHT.
08 Aplicación en el mundo real	Extraer el dominio de un correo, el código de área de un teléfono o el año de una fecha guardada como texto.

Ejercicio 31: COALESCE

01 Ejercicio	Ejercicio 31: COALESCE
02 Objetivo	Encontrar el arma de cada luchador, dando prioridad al tanque, luego al arma de fuego y por último a la espada.
03 Concepto SQL	COALESCE devuelve el primer valor no nulo de una lista de columnas.
04 Consulta ejecutada	`SELECT name, COALESCE(tank, gun, sword) AS weapon` `FROM fighters;`
05 Resultado	4 filas con name y weapon 
06 Interpretación	Devuelve 4 luchadores: US Marine con M1A1 Abrams Tank, John Wilkes Booth con .44 caliber Derringer, Zorro con Sword of Zorro, e Innocent Bystander con NULL por no tener ninguna de las tres armas.
07 Aprendizaje	El orden de las columnas dentro de COALESCE define la prioridad. Si todas son NULL, el resultado también lo es. AS weapon da nombre a la columna calculada.
08 Aplicación en el mundo real	Mostrar el teléfono móvil de un cliente y, si no existe, el fijo; o reemplazar valores vacíos por un dato alternativo en un reporte.